

JWALKER Knowledge Assistant

Grounded Retrieval Assistant for a WordPress Membership Site - Comprehensive Technical Overview

A client-readable and engineer-ready reference for a members-only knowledge assistant: architecture, hybrid retrieval, grounding and refusal behavior, source synchronization, signed member authentication, restricted internet search, isolated deployment, verification evidence, operational lessons, and current verified limitations.

**Python | FastAPI | SQLite + FTS5 | NumPy Cosine | Reciprocal Rank Fusion | Pluggable
Embeddings | Google Gemini | HMAC-Signed WordPress Tokens | systemd + nginx | Docker
(optional)**

Public technical edition · Evidence reconciled through 15 July 2026

1. Reading Guide and Evidence Policy

This document serves two audiences at once. Business readers can stay with the executive overview, product boundaries, current verified state, before-and-after evidence, limitations, and status summary. Engineers can continue into the request lifecycle, retrieval algorithms, synchronization contracts, authentication design, failure semantics, deployment discipline, and representative traces.

Claims are based on the current integrated source tree, the dated local verification report, the acceptance-criteria record, and the project defect log. Where a capability is implemented and locally verified but has not been proven against live client-controlled infrastructure, those two states are reported separately rather than merged into a single completion claim.

The content is intentionally safe for broad professional review. It describes system behavior and engineering decisions without exposing passwords, API keys, tokens, private SSH key material, server addresses, hostnames, database names, account identifiers, private file paths, or the identity and private content of the end content owner.

Truth conventions used throughout:

- **Current implementation** - behavior confirmed in the integrated source and the most recent local verification record.

- **Locally verified** - a dated result produced by an offline or fixture-backed test run, not a live production proof.
- **Access-gated** - an implemented capability whose live verification requires credentials or infrastructure controlled by the client.
- **Known limitation** - a verified boundary or missing gate, stated directly rather than hidden behind a broad production-ready claim.

2. Executive Overview

The JWALKER Knowledge Assistant lets members of a WordPress membership site ask a real question and receive an answer drawn from the creator's own material rather than the open internet. A member types a question into a chat interface embedded on a members-only page. The engine embeds the question, retrieves matching passages from a curated knowledge base built from the creator's content, and asks a language model to answer using only those passages.

Grounding is not left to the prompt. A relevance gate runs on the retrieved passages before generation, and when nothing clears the threshold the assistant returns a configured refusal instead of an invented answer. The rules that forbid fabrication live in code and cannot be removed by editing the assistant's configurable personality.

The retrieval path is hybrid rather than pure vector. Dense semantic similarity is computed in NumPy over embeddings stored in a single SQLite file, keyword matching uses SQLite's FTS5 full-text index over the same rows, and the two rankings are combined with Reciprocal Rank Fusion. Pure vector search reliably misses exact terms such as a program name or a job title; the keyword arm recovers them.

Four content connectors feed the knowledge base: a public YouTube transcript path, a Google Drive service-account path, a Patreon path, and a Kajabi path. Three of the four synchronize a complete snapshot atomically, so a re-index adds new documents, replaces edited documents, removes deleted documents, and leaves the last good database intact when discovery fails.

A separate restricted internet search exists as a controlled query-time source. It is not ingested, and it fails closed: it cannot run unless the operator has supplied both an approved keyword allowlist and an approved list of HTTPS sites.

Current high-level characteristics:

- **Grounded answers with an honest refusal:** cosine relevance gating, a locked no-fabrication instruction, and a configurable refusal message with source citations on every grounded reply.

- **Hybrid retrieval without a vector database:** dense plus keyword ranking fused by RRF over one portable SQLite file, with no external database server and no per-seat or per-month vector service.
- **Pluggable embeddings:** a local CPU model as the zero-cost production default and a hosted provider as a torch-free alternative, selected by configuration.
- **Members-only by construction:** the WordPress plugin renders nothing for logged-out visitors and mints a short-lived HMAC-signed token; the long-lived signing secret never reaches browser JavaScript.
- **Atomic source synchronization:** discovery, extraction, and embedding all complete before a single database transaction, so a connector or provider failure cannot destroy the working index.
- **Isolated deployment posture:** the target server already runs six unrelated agents, so every deployment artifact is additive - a dedicated user, directory, virtual environment, local port, new service units, and a new proxy block.

3. Product Purpose, Users, and Boundaries

The assistant is a retrieval and navigation layer over a body of proprietary content. Its job is to let a member ask a plain question and get a direct answer, with links back to the specific source material, instead of manually searching hours of video and scattered posts across several platforms.

Primary users:

- **Members** of the WordPress site, who ask questions through the embedded chat interface and receive grounded answers with sources.
- **The content owner**, whose material forms the knowledge base and whose voice the assistant's configurable persona is tuned to match.
- **The operator**, who runs connectors, reviews answer quality, and controls which sources are enabled.
- **Engineers**, who maintain retrieval behavior, connector contracts, deployment health, and regression evidence.

Explicit non-goals:

- No answers from the open internet. Web results are restricted by an approved keyword gate and an approved site allowlist, and are labelled as web-sourced when shown.
- No invented facts, names, links, numbers, or quotes. When the retrieved content does not answer the question, the assistant says so.
- No guarantee that a generated answer is complete or current for an individual member's situation.
- No automatic content refresh from a source that only offers an interactive login with two-factor authentication. A background job cannot type a texted code.

- No claim that a locally verified capability has been proven in production. Live criteria are tracked separately.

4. Starting Point and Engineering Constraints

The creator's material was spread across five silos: two public YouTube channels, a membership platform, a course platform, a cloud drive of reference files, and selected reference material on the open web. Members had no way to query any of it. Answering a straightforward question meant hunting through long-form video and scattered posts.

Three client constraints shaped every subsequent architecture decision:

- **No recurring subscription costs.** No paid plugins, no hosted vector database, no flat monthly SaaS. The only acceptable running costs were a small server and pay-as-you-go model usage on the client's own account.
- **Answers must stay grounded** in the creator's own content, with internet search restricted rather than open.
- **Members-only access** at go-live.

Two infrastructure facts emerged during the first phase and materially changed the plan. The WordPress site runs on shared managed hosting that serves PHP only, so the engine could never live where the site lives. Later, the deployment target became an existing cloud instance already running six unrelated agents, with an explicit instruction not to disturb them. That converted deployment from a routine install into an isolation problem.

5. Current Verified State

The reference point is the integrated source tree verified on 15 July 2026 together with the recorded local verification run.

- **Engine:** a single FastAPI process exposing liveness, store statistics, ingestion, and chat routes. It refuses to start when production configuration is unsafe.
- **Knowledge store:** one SQLite file holding chunk text, source metadata, stable source keys, content hashes, and embedding blobs, with an FTS5 companion index and a LIKE fallback where FTS5 is unavailable.
- **Packaged data:** 228 transcript chunks from 10 distinct video URLs, ingested from the two public channels.
- **Retrieval:** dense cosine plus keyword ranking fused with RRF, an optional source-type filter, and a relevance floor feeding the refusal guardrail.
- **Generation:** Gemini Flash by default, with a retrieval-only stub mode when no key is configured so the whole pipeline stays testable before provider billing exists.
- **Connectors:** YouTube is additive by stable video URL; Drive, Patreon, and Kajabi synchronize complete snapshots with add, edit, and delete handling. All connector

contracts are fixture-backed and locally verified. None has been run against real client data.

- **Authentication:** signed member tokens, a separate ingestion secret, and per-client rate limiting are implemented and covered by HTTP behavior tests.
- **Restricted web search:** implemented, disabled by default, and fails closed. The client's approved keyword list, site list, and fallback-versus-equal mode decision remain outstanding.
- **Deployment:** isolated service, timer, and proxy templates render from configuration and are locally verified. No production server has been modified.

6. System Topology

The assistant is a small system with clear ownership boundaries. WordPress owns member identity. The engine owns retrieval, grounding, and generation. Connectors own content discovery and extraction. Nothing shares a database with the website.

Member browser (WordPress members-only page)

|

| chat widget + short-lived signed token

v

Reverse proxy (new server block, own hostname)

|

v

FastAPI engine process (dedicated user, localhost bind)

|

+--> auth guard + rate limit

+--> query embedding

+--> hybrid retrieval <--> SQLite knowledge file (+ FTS5)

+--> relevance gate

+--> restricted web search (gated, optional)

+--> grounded generation

|

v

Answer + deduplicated source citations

Scheduled timers --> connectors --> discovery/extraction/embedding --> atomic sync

Scheduled timers --> online backup --> retention pruning

Primary boundaries:

- **WordPress:** authenticates the member, renders the widget only when logged in, and mints a short-lived signed token. It never holds knowledge data.
- **Engine process:** binds to localhost and is reached only through the reverse proxy. It is the only component that reads the knowledge file.
- **Knowledge file:** a single SQLite database, portable as one file, backed up by an online-backup job rather than a file copy.
- **Model provider:** query embedding when the hosted embedding provider is selected, answer generation, and grounded web search. It never receives credentials belonging to any content platform.
- **Content platforms:** read-only access only. The Drive integration uses a read-only scope and never modifies the client's files.

7. End-to-End Request Lifecycle

The chat path is ordered so that cheap deterministic checks happen before any paid or network-heavy operation.

- 1 Authenticate: signed member token, or direct key, or open local mode
- 2 Rate-limit the resolved client identity
- 3 Reject an over-length question
- 4 Validate any requested source-type filter against the supported set
- 5 Embed the question with the configured provider
- 6 Retrieve: dense ranking + keyword ranking, fused by RRF

- 7 Apply the relevance floor to produce the grounded passage set
- 8 Optionally run restricted web search, per gate and mode
- 9 Generate the answer from the passage set, or return the configured refusal
- 10 Deduplicate sources and return answer, grounded flag, sources, and model

Each early exit is deliberate. An unauthenticated request never reaches embedding. A rate-limited request never reaches the provider. A question with no relevant passages never reaches generation at all, because the refusal is produced without a model call. This keeps both cost and the number of states in which a model can influence a response as small as possible.

The ingestion path is separate and runs under its own secret:

Discover source items -> extract text -> chunk on word boundaries

-> hash each document -> skip unchanged documents -> embed changed documents

-> single atomic transaction: delete stale rows, insert replacements

8. Knowledge Store Design

The knowledge store is one SQLite file. This was a deliberate downgrade from an earlier design that used a dedicated vector database and container orchestration.

At this corpus size, an in-process store is faster to operate, free to run, and portable: the entire knowledge base is a single file that can be copied, backed up, or handed over. A separate vector database is not free. It is another process to run, secure, monitor, and pay for, and it conflicted directly with the client's no-subscription constraint.

Schema responsibilities:

- **Chunk text** and its **embedding blob**, stored as float32 bytes.
- **Source type** ([youtube](#), [drive](#), [patreon](#), [kajabi](#)), used for filtering and statistics.
- **Title** and **URL**, used for the citations shown with each answer.
- **Source key** - a stable per-document identity within a source, which is what makes edit and delete synchronization possible.
- **Content hash** - a digest of the extracted document text, used to skip unchanged documents without re-embedding them.

The FTS5 virtual table is an external-content index over the same rows. FTS5 ships with most SQLite builds but not all, so availability is detected once at startup; when it is absent, keyword search degrades to a bounded LIKE query rather than failing. Schema migration is additive: the store detects missing columns on an older database, adds them, creates the source-key index, and rebuilds the full-text index when it did not previously exist.

Embeddings are loaded into one matrix and cached in memory, with the cache invalidated on every write path. Because vectors are normalized at creation, cosine similarity is a plain dot product against that matrix.

9. Embedding Providers and the Dimension Contract

Embeddings are pluggable behind one configuration value.

- **Local provider:** a small CPU sentence-transformer model producing 384-dimensional vectors. This is the documented production default because it costs nothing per query and consumes no provider quota.
- **Hosted provider:** a hosted embedding API producing 768-dimensional vectors by default. It requires no heavy machine-learning dependency, which keeps the container image small and lets the whole pipeline build and test on toolchains where the local model cannot be installed.

Both providers return L2-normalized vectors so the store can treat cosine similarity as a dot product, and both embed documents and queries with the matching task type where the provider supports it.

The dimension contract is the important part. Changing provider changes vector length. Vectors from two providers must never coexist in one database, and the knowledge base must be rebuilt after a provider change. This is stated in configuration comments, in the example environment file, and in the handover document, because it is the kind of change that produces silently wrong retrieval rather than a loud failure.

The hosted provider path also carries explicit quota discipline: bounded batch sizes, deliberate pacing between batches, and retry backoff that honors the server-supplied retry delay rather than a fixed guess.

10. Hybrid Retrieval and Fusion

Retrieval runs two independent rankings over the same candidate pool and fuses them.

- **Dense ranking:** the query vector is multiplied against the cached embedding matrix, scores are sorted, and the top candidates are returned with their cosine scores.

- **Keyword ranking:** the raw question text is matched through FTS5 with its own ordering, optionally constrained by source type. A malformed full-text query falls through to a bounded LIKE search rather than raising.
- **Fusion:** Reciprocal Rank Fusion combines both lists, giving each candidate a contribution of $1/(k + \text{rank})$ from each list it appears in, with the conventional constant. Candidates appearing in both lists rise; candidates strong in only one still compete.

The candidate pool is deliberately wider than the requested result count so fusion has material to work with, and the requested result count is clamped to a configured maximum so a client cannot request an unbounded context window.

The cosine score, not the fused score, is carried onto each returned passage. This matters: the fused score is a rank-combination artifact with no absolute meaning, whereas cosine similarity is the signal the relevance gate needs. Using the fused score as a relevance threshold would be meaningless.

Source filtering is applied inside both rankings rather than after fusion, so a filtered query genuinely searches within the selected sources instead of filtering an already-truncated global result set.

11. Relevance Gating and the Refusal Guardrail

Retrieval always returns its best candidates. Best is not the same as relevant. The relevance gate is what separates the two.

Every retrieved passage carries a cosine score. Passages below the configured floor are discarded. If nothing survives, the assistant returns the configured refusal message and no sources, and the response is marked as not grounded. No model call is made.

The threshold is embedding-model-specific and is treated as such. Different embedding models sit on different cosine floors: a score that means "unrelated" for one model can mean "loosely related" for another. The floor is therefore documented per provider, with the value chosen by measuring the actual score distribution of known-relevant and known-irrelevant query pairs rather than by picking a round number.

The refusal message itself is configuration, not code. It carries voice and wording the client may want to tune, so it sits alongside the persona and temperature as an editable answer tunable.

12. Answer Generation and the Persona and Rules Split

The system prompt is built from two parts with different trust levels.

- **Persona** is configuration. It sets voice, warmth, and register. The client can rewrite it freely without a code change.
- **Grounding rules** are locked in code and appended after the persona. They require the model to answer only from the supplied content, to say so honestly when the answer is not present, and never to invent facts, names, links, numbers, or quotes.

This split exists so that a tone request can never become a safety regression. A client editing the persona to sound friendlier cannot accidentally delete the instruction that prevents fabrication.

Retrieved passages are assembled into a numbered, labelled context block so the model can attribute what it uses. Generation runs at a configured temperature, warm enough to sound human and low enough to stay disciplined.

When no provider key is configured, the engine runs in **stub mode** and returns the top retrieved passage with an explicit stub label instead of a generated answer. This keeps the full pipeline standable and testable before provider billing exists, which is exactly the situation the project spent most of its life in.

13. Restricted Internet Search

Restricted web search is a query-time source. It is never ingested into the knowledge base, so it can never contaminate the creator's corpus or persist a third-party claim as if it were the creator's own material.

It is gated four ways and is a complete no-op unless every gate passes:

1. **Explicitly enabled** in configuration. Default is off.
2. **Keyword gate.** The question must contain at least one client-approved keyword, matched on word boundaries. An empty allowlist means nothing is approved, so the gate returns false. This is the deliberate fail-closed behavior: an unfilled list can never silently become open-web search.
3. **Site allowlist.** Every configured site must match a strict domain-with-optional-path pattern. This rejects search-operator injection through the site list.
4. **Result validation.** Every citation URL returned by the provider is re-checked against the allowlist: HTTPS scheme only, host equal to or a subdomain of an approved domain, and path prefix respected. A URL outside the list is dropped even though the provider returned it.

Two modes are supported. **Fallback** runs web search only when the creator's own content produced nothing relevant, and the answer is prefixed with an explicit note that it did not come from the creator's content. **Equal** always attempts the web as well, keeps the content-grounded

answer primary, and appends the web citations alongside. Both modes preserve honest provenance by tagging web results with a distinct source type.

The mode decision, keyword list, and site list are client decisions that remain outstanding. Until they arrive, the feature stays off and the code path is inert.

14. Source Connectors and Content Coverage

Four connectors feed the knowledge base, each with a documented access reality.

- **YouTube.** Channel handles resolve to channel identifiers from the public channel page, recent videos come from the public feed, and transcripts come from a transcript library. No platform API key is required. This is the only source that refreshes automatically without client-controlled credentials.
- **Google Drive.** A service-account key with a read-only scope walks nested folders recursively, downloads supported files, and exports native document formats to text. A service account is used specifically because it is headless: a scheduled job can run it without an interactive login.
- **Patreon.** Either a creator access token against the platform's v2 campaign posts API, or a complete export folder on disk.
- **Kajabi.** Either a bearer token against the official courses, modules, and lessons API, or a complete export folder on disk. Export paths become readable titles of the form `Course / Module / Lesson`.

Every network path in every connector enforces HTTPS and validates the target host against the configured API base before making a request, so no connector can be redirected to an arbitrary scheme or host.

The automation constraint is stated plainly rather than glossed over. Automatic refresh requires machine-level access: an API token or a service account. A source that offers only an interactive login protected by two-factor authentication cannot be refreshed by a background job, because a scheduled process cannot receive and enter a one-time code. Such a source stays current only through periodic manual export. This is a property of the platform, not of the implementation, and it must not be promised away.

15. Atomic Source Synchronization

Adding content is easy. Reflecting edits and deletions is where naive ingestion breaks, and it is the difference between a demo and a maintainable index.

Each synchronized document carries a **stable source key** - a Drive file identifier, an export-relative path, a post or lesson identifier - and a **content hash** of its extracted text. The synchronization routine:

1. Requires discovery and extraction to finish completely before touching the database.
2. Rejects duplicate or missing source keys outright.
3. Loads the stored manifest of source key to content hash for that source type.
4. Skips every document whose hash is unchanged, so unchanged content is never re-embedded and never re-billed.
5. Embeds only the changed documents, still outside the transaction.
6. Opens one immediate transaction, deletes rows for documents that are gone or being replaced, inserts the replacements, and commits. Any failure rolls back.

The ordering is the safety property. Because discovery, extraction, and embedding all complete before the transaction opens, a connector failure or a provider outage leaves the last good knowledge base fully intact. The index is never emptied in the hope that a refill will succeed.

YouTube is handled differently and deliberately: it is additive by stable video URL. The public feed exposes only a window of recent videos, so treating a feed response as a complete snapshot would delete every older video the moment it aged out. Additive ingestion with skip-if-known makes re-runs idempotent, which removes the only reason anyone would ever be tempted to wipe the database.

One boundary carries a warning in the handover: an export connector treats its folder as the complete snapshot. Pointing it at a partial temporary folder will intentionally remove the documents that folder is missing.

16. Text Extraction Contract

All connectors share one extraction module, so parsing is tested once rather than reinvented per source. It converts plain text, Markdown, HTML, PDF, and Word documents into clean text, dispatching on file extension and returning an empty string for unsupported or empty documents so connectors can skip them cleanly.

The module performs no network access at all, which makes it fully unit-testable offline against local fixtures. Heavy parsers are imported lazily so importing the module never pulls them into memory, and HTML extraction strips script, style, and noscript content before taking text.

Chunking is shared with the same word-boundary chunker used by transcript ingestion, at a configured target size with a configured overlap so context is not cut mid-thought.

17. Authentication: WordPress Signed Member Tokens

Client-side gating is not security. Rendering a widget only for logged-in users prevents a casual visitor from seeing it; it does nothing to stop a direct request to the engine. The engine therefore enforces its own boundary.

The design:

- The WordPress plugin renders nothing at all for logged-out visitors, and nothing when its required configuration constants are absent. It fails closed in both directions.
- For an authenticated member, the plugin mints a token containing the member subject, an issued-at time, and an expiry, signed with HMAC-SHA256 using a shared secret defined in the site configuration file.
- The secret itself is never emitted into HTML or JavaScript. Only the short-lived signed token reaches the browser.
- The engine verifies the signature in constant time, then validates the subject, the issue time against permitted clock skew, the expiry, and the token lifetime against the configured maximum. A token claiming a longer life than the configured ceiling is rejected even if its signature is valid.
- Ingestion is protected by a **separate** secret, so a member token can never reach the route that writes to the knowledge base.
- Both secrets default to empty, which leaves the endpoint open. That is acceptable for local development and is explicitly rejected by production configuration validation.

The plugin escapes every attribute it renders, and the engine validates question length and requested source types before doing any work.

18. Rate Limiting and Cost Control

Every chat request that reaches generation costs money on the client's provider account. Rate limiting is therefore a cost control as much as an abuse control.

The engine applies a sliding one-minute window per resolved client identity, where identity is the authenticated token subject when available and the network peer otherwise. Exceeding the configured limit returns HTTP 429. Setting the limit to zero disables it.

This is an in-process limiter, which is correct and sufficient for a single-process deployment behind one reverse proxy. A multi-worker deployment would need to move the window into a shared store, and that is recorded as a known limitation rather than left implicit.

Provider budget caps and alerts are a separate, account-owner responsibility. Budget alerts notify; they do not automatically stop usage. No component of this system enables billing, provisions infrastructure, or changes a paid plan.

19. Configuration Boundary and Production Validation

Nothing that varies between deployments is hardcoded. Model identifiers, provider selection, thresholds, chunk sizes, result limits, timeouts, retry pacing, allowed origins, source toggles, API base URLs, deployment paths, ports, retention windows, the assistant persona, the refusal message, and every secret all live in one typed settings object populated from the environment. An example environment file documents every variable with safe placeholders and carries no real values.

The only secret with a default is none: the provider key defaults to empty, which puts the engine in retrieval-only stub mode so the entire stack remains standable and testable without any credential at all.

Production validation is enforced at startup, not documented as advice. When the environment is marked as production, the engine refuses to start if any of the following is true:

- Allowed origins are still the permissive wildcard rather than the exact site origin.
- The member shared secret is missing.
- The ingestion secret is missing.
- The provider key is missing.
- The deployment server name is missing.
- Web search is enabled without both an approved keyword list and an approved site list.
- The web search mode is not one of the two supported values.

Local and fixture modes stay permissive, so development is unobstructed while an unsafe production launch is impossible rather than merely discouraged.

20. Isolated Deployment Design

The deployment target is an existing server already running six unrelated agents, with an explicit instruction not to disturb them. That constraint drives the entire deployment design toward being purely additive.

Deployment artifacts:

- A **service unit** running under a dedicated non-root user and group, from a dedicated application directory and its own virtual environment, bound to localhost only. Hardening includes no privilege escalation, a private temporary directory, a strict read-only system view, and a single writable path for the data directory.

- A **new proxy server block** on its own hostname, forwarding to the local port. It is never an edit to an existing block.
- **Timer and one-shot units** for scheduled re-indexing and daily backup.
- A **template renderer** that substitutes the application directory, user, port, and server name from configuration, so no environment-specific value is committed in a template.

The deployment discipline is explicit: take a read-only inventory of the existing agents first, capture their state, ports, resources, and proxy configuration, and compare any existing assistant files against local copies before writing anything. Stop if the intended port is occupied or resources are insufficient. Install only the newly rendered units. Never install globally, upgrade the operating system, edit another service, or restart another agent. Validate the proxy configuration before reload. After deployment, hash every deployed file against its local counterpart and re-check all six pre-existing agents against the inventory.

Container packaging exists and is verified as an option, but it is not required. The runtime remains a single Python process with a file-based store.

21. Scheduled Re-indexing, Backups, and Restore

A daily timer runs the re-index one-shot across the configured sources. Which sources refresh is configuration, so a source without machine-level access can be excluded without a code change.

Backups use SQLite's online backup API rather than a file copy. This matters: copying a database file while a process may be writing to it can produce a torn backup, whereas the online backup API produces a consistent snapshot. Backups are timestamped in UTC, and files older than the configured retention window are pruned on each run, with the current backup never eligible for pruning.

Restore is a scoped, four-step procedure: stop only the assistant service, copy the selected backup over the database path, preserve ownership, start only the assistant service, then verify the store health route and one grounded question. No other service is stopped at any point.

22. Release Packaging and Secret Audit

The client package is built from an **explicit allowlist**, not from a directory sweep with exclusions. This inversion is deliberate: an allowlist fails safe when a new file appears, whereas a denylist silently ships whatever nobody thought to exclude.

The builder enumerates named files and a small set of globs covering application code, connectors, deployment templates, web assets, and the WordPress plugin. Before writing the archive, it audits every candidate:

- Rejects forbidden filenames outright: the environment file, the credentials file, the private dossier, and the internal tracker.
- Rejects anything under the key directory.
- Scans file contents for private key blocks, cloud access key patterns, provider API key patterns, source-control token patterns, and any non-empty assignment to a known secret environment variable.

Any finding aborts the build. The resulting archive contains no credentials, no private keys, no environment file, no local virtual environment, no logs, and no internal project notes. Archive integrity, forbidden-path auditing, and an independent secret scan are all part of the recorded verification run.

23. Test Strategy

23.1 Connector and extraction

Fixture-backed tests cover text, Markdown, HTML, PDF, and Word extraction; export-folder traversal for both membership and course sources; recursive Drive discovery; course, module, and lesson traversal against the official API contract; and pagination behavior.

23.2 Synchronization

Two-run tests prove the behavior that matters: a second run with no changes re-embeds nothing, an edited document is replaced, a deleted document is removed, and a failed discovery leaves stored data intact.

23.3 Retrieval and grounding

Tests cover hybrid ranking correctness, source-type filtering, the relevance floor, refusal behavior on an off-topic question, restricted-web provenance labelling in both modes, and end-to-end ingest through answer.

23.4 Security and HTTP behavior

Tests cover missing, expired, malformed, and invalid signed tokens; token lifetime beyond the configured ceiling; ingestion-secret separation from the member secret; rate-limit enforcement returning 429; over-length questions; unsupported source-type filters; and production-configuration rejection.

23.5 Operations

Tests cover backup creation and retention pruning, deployment template rendering, and migration of a legacy database to the current schema including full-text index rebuild.

23.6 Static and supply-chain gates

Lint, type checking, security scanning, a rules-based security scan, dependency consistency, JavaScript syntax, container build and health, and a secret audit of both the configuration surface and the final archive.

24. Verified Test Evidence and How to Read It

Passed, as of the 15 July 2026 local verification run:

- Eleven offline integration and behavior scripts covering connector parsing, hybrid retrieval, atomic synchronization, recursive Drive discovery, membership and course API contracts, production configuration, backup and retention, signed HTTP authentication, ingestion-secret separation, rate limiting, source filters, restricted-web provenance, deployment rendering, release and configuration auditing, and legacy database migration.
- Dependency consistency with no broken requirements.
- Lint, type checking with a declared missing-stub allowance for optional third-party packages, security scanning with zero findings and zero suppressions, a rules-based Python and OWASP scan, and JavaScript syntax checks.
- A fresh container image build and quiet compose validation. A temporary isolated container returned success from both health routes, and all source connectors imported successfully.
- Release archive integrity, forbidden-path audit, and an independent secret scan.
- Packaged knowledge base: 228 chunks from 10 distinct video URLs.

Explicitly not claimed as passed:

- Live server inventory and deployment. The provided cloud login cannot enumerate instances, and no usable address, login user, or installed key has been supplied.
- Live Drive, membership, course, and restricted-web pulls. Machine access, exports, folder identifiers, approved lists, and the mode decision have not been supplied.
- The production WordPress logged-out and logged-in flow over HTTPS, which requires both the live engine and a site change.
- PHP syntax validation of the plugin, because a PHP command-line interpreter was not available in the verification environment.
- An independent fresh-context review of the final diff, which remains a pre-release requirement.
- A live provider rerun, intentionally skipped after a development credential was exposed during container configuration validation. That credential requires rotation before any further provider test.

How to read these totals. They describe one dated verification run of the local package. They are not a claim that the deployed system has been proven in production, and they must be re-run after any change to retrieval, connectors, configuration, or dependencies.

25. Before and After

- **Content access - before:** five disconnected silos with no query surface; answering one question meant manually hunting through long-form video and scattered posts.
- **Content access - after:** one members-only chat entry point over a unified knowledge base with per-answer source citations.
- **Retrieval - before:** none.
- **Retrieval - after:** hybrid dense and keyword ranking fused by RRF, with optional source-type filtering and a measured relevance floor.
- **Architecture - before:** a container-orchestrated stack with a dedicated vector database, sized far above the corpus and against the client's cost constraint.
- **Architecture - after:** one Python process and one portable SQLite file, with containers retained as an optional packaging path rather than a requirement.
- **Grounding - before:** no gate between retrieval output and the model.
- **Grounding - after:** a per-provider cosine floor, a configured refusal returned without a model call, and no-fabrication rules locked in code beneath an editable persona.
- **Refresh behavior - before:** transcript ingestion could only add, and re-running risked duplicates.
- **Refresh behavior - after:** idempotent additive transcript ingestion plus atomic add, edit, and delete synchronization for the three snapshot sources.
- **Failure behavior - before:** a transcript provider block was swallowed and reported as "no transcript", which could empty the store silently.
- **Failure behavior - after:** genuine absence is distinguished from a provider block; a block stops the run loudly and leaves stored data untouched.
- **Access control - before:** open endpoints.
- **Access control - after:** signed short-lived member tokens, a separate ingestion secret, per-client rate limiting, and startup refusal on unsafe production configuration.
- **Handover - before:** no secret-safe package existed.
- **Handover - after:** an allowlist-built archive with a content audit and an independent secret scan.

26. Incident Chronology and Engineering Lessons

These incidents are recorded because they explain why specific gates exist. Details are limited to reusable engineering lessons; private operational information is excluded.

26.1 The engine could not live where the website lives

The original plan assumed the assistant could run alongside WordPress. The site runs on shared managed hosting that serves PHP only, with no long-running process capability. The architecture split into a WordPress front door and a separately hosted engine. **Lesson:** confirm the host type before promising where an always-on service will run. "It is on WordPress" is not a deployment target for a Python service.

26.2 The first architecture was oversized for the job

The initial build used a dedicated vector database and container orchestration for a modest corpus, chosen out of habit rather than sized to the work or to the client's explicit no-subscription constraint. It collapsed to a single process with a file-based hybrid store, re-verified end to end. **Lesson:** match the stack to the corpus and the cost constraint. A separate vector database server is not free; it is another box to run, secure, and pay for.

26.3 A relevance threshold was carried across embedding models

After switching embedding providers, an off-topic question returned a grounded response with sources even though the model itself correctly refused. The relevance floor had been tuned for the original model; the replacement model's cosine distribution sat much higher, so unrelated pairs cleared the old gate. The fix measured the actual score distributions for known-relevant and known-irrelevant pairs and set a provider-specific floor, documented per provider. **Lesson:** a similarity threshold is a property of the embedding model, not of the system. Measure the distribution; never inherit another model's number.

26.4 A heavy dependency blocked the build environment

The agreed local embedding path could not be installed in the working environment because a very recent language runtime outpaced the machine-learning library's builds, and the dependency was large for a quick demo besides. Embeddings became pluggable behind a configuration value, so the pipeline builds and tests anywhere while the local path remains the documented zero-cost production default. **Lesson:** put the heavy or fragile dependency behind a provider interface so a toolchain gap can never block the build, and you keep an escape hatch for production.

26.5 Good data was deleted before the replacement was verified

To grow the demo corpus, the knowledge database was cleared and ingestion re-run. Every fetch returned "no transcript" and the store was left empty. Two mistakes compounded: the ingester swallowed all transcript errors into a single "no transcript" outcome, hiding that the real error was a provider IP block; and the working data was deleted before the replacement was in hand. The fix distinguished genuine absence from a block, made the block raise and stop the run loudly with the database untouched, and made ingestion idempotent by recording ingested

URLs so there is never a reason to wipe the store. **Lesson:** never delete good data before the replacement is verified, and never let a pipeline swallow a third-party failure into a success-shaped empty result.

26.6 Two independent free-tier limits collided

Ingesting the full corpus failed on an embedding quota, and re-running to recover re-fetched every transcript and triggered the transcript provider's burst block. Two rate limits from two providers produced one empty database. The fix batched and paced embedding requests with backoff honoring the server-supplied retry delay, cached each transcript to disk the instant it was fetched so an embedding retry never touches the transcript provider again, and added a deliberate delay between transcript fetches. **Lesson:** persist the expensive-to-fetch step before the rate-limited step, pace both, and never couple them so one's failure re-triggers the other.

26.7 A user-facing string stayed hardcoded after a configuration pass

An editable persona was added so the client could tune the assistant's voice. The refusal message shown when nothing relevant is found remained hardcoded, even though it carries exactly the same voice and wording the client would want to tune. It moved to configuration alongside the persona and temperature. **Lesson:** externalizing configuration means every string a client might reword, not just the headline one. When you touch one user-facing message, scan the whole function for its siblings.

26.8 The browser widgets did not send the credential the protected route required

Protected chat was implemented server-side, but the browser widgets were not updated to present the credential, so the real end-to-end flow failed while unit tests passed. The fix added signed-token HTTP tests and a WordPress integration acceptance criterion. **Lesson:** an authentication change is not complete until the real client flow is exercised. Server-side tests prove the guard; they do not prove the caller.

26.9 Connectors skipped known URLs and therefore could not apply edits or deletions

Skip-if-known made re-runs cheap but also made edits and deletions invisible, because a changed document at a known URL was simply skipped. The fix introduced stable source keys and content hashes, and replaced skip-based ingestion with atomic snapshot synchronization for the three sources where a complete snapshot is meaningful. **Lesson:** idempotency by URL is not synchronization. Reflecting edits and deletions requires a stable document identity and a content digest.

26.10 A placeholder API path sat behind a "complete" status

A course connector's API path was a deliberate placeholder while the project status claimed all connector code was done. It was rebuilt against the platform's official documented API and covered by a mocked contract test. **Lesson:** a placeholder is a defect the moment status says otherwise. Implementation status must be reported per code path, not per module.

26.11 A container configuration command expanded and printed a development credential

A container configuration validation command was treated as a structural syntax check. It expands environment files by default, so it printed a resolved development credential into tool output. The credential is treated as compromised and requires rotation; future validation uses a generated fake environment file or quiet output and never prints resolved secrets. **Lesson:** a command that reads an environment file is a secret-expansion operation, not a syntax check. Review commands for what they read before running them, not only for what they change.

26.12 The managed host rejected a name-derived administrator password

A password derived from the account and site name was rejected by the host's personal-information and strength checks. It was replaced with a strong non-derived value. **Lesson:** never derive a credential from the client, the brand, or the site name. Managed hosts reject it, and it is a weak practice regardless of whether they do.

27. Known Limitations and Risk Register

- **Live production has not been proven.** Deployment, live source pulls, and the production WordPress flow are all blocked on client-controlled access. The local package is complete and verified; the deployed system is not, and must not be represented as such.
- **A development credential was exposed and requires rotation** before any further provider or production use. It must not be deployed or reused.
- **The rate limiter is in-process.** It is correct behind a single process and one reverse proxy. A multi-worker deployment would need a shared store, or the limit becomes per-worker.
- **The plugin has not passed a PHP syntax and runtime check,** because no PHP interpreter was available in the verification environment. This remains a pre-release item.
- **No independent fresh-context review** of the final diff has been performed. It is required before production sign-off.
- **Export connectors treat their folder as the complete snapshot.** Pointing one at a partial folder will intentionally delete the documents it is missing.

- **Sources without machine-level access cannot auto-refresh.** A background job cannot satisfy interactive two-factor authentication. Those sources stay current only by manual re-export, and this must not be promised away.
- **Switching embedding provider invalidates the index.** Vector dimensions change, and mixing providers in one database produces silently wrong retrieval rather than a loud failure.
- **The relevance floor is a threshold, not a calibrated confidence.** It is a measured, useful control, not a probability that an answer is correct.
- **The transcript path depends on an unofficial provider surface** that can rate-limit or IP-block. The system now fails loudly and preserves data, but it cannot make the block go away.
- **The corpus is a documented snapshot.** 228 chunks from 10 videos describes the verified packaged state, not a permanent content guarantee.
- **Restricted web search is unconfigured.** The approved keyword list, site list, and mode decision are outstanding, and the feature stays inert until they arrive.
- **The full-text index degrades to a bounded LIKE scan** where FTS5 is unavailable, which changes keyword ranking quality on such a build.
- **Generated answers still require human review** for claims, currency, and tone, particularly where content touches money, employment, or outcomes.

28. Recommended Next Engineering Gates

1. Rotate the exposed development credential and configure a new production credential only through the server environment.
2. Complete the read-only server inventory of the six existing agents, then deploy additively and prove file-level parity and agent non-disruption afterward.
3. Run the production WordPress flow end to end over HTTPS, logged out and logged in, including expired-token rejection.
4. Obtain the restricted-web keyword list, site list, and mode decision, then verify both modes against live provenance.
5. Add a PHP syntax and runtime check to the gate set so the plugin is never shipped unverified.
6. Obtain an independent fresh-context review of the final diff against the acceptance criteria.
7. Move rate limiting into a shared store before any multi-worker deployment.
8. Add read-back verification after each connector run: compare expected document counts against stored counts and report drift.
9. Introduce corpus release metadata recording content version, chunk count, ingestion date, embedding provider, and dimension for every knowledge release.
10. Add a golden retrieval set of fixed questions with validated chunk identifiers, so a threshold, provider, or chunking change cannot silently alter the evidence behind answers.

11. Aggregate request-stage timings into percentiles by route and cache state rather than relying on single observations.
12. Configure provider budget caps and alerts on the account owner's side, with an explicit acknowledgement that alerts notify rather than stop usage.

29. Technology Stack and Responsibilities

29.1 Application

- **Python and FastAPI** - routes, dependency-injected auth guards, lifespan configuration validation, and orchestration.
- **Pydantic settings** - one typed configuration surface populated from the environment.
- **Browser JavaScript** - the chat widget and search-bar widget, served by the engine so a single URL provides both the API and a working interface.
- **WordPress plugin (PHP)** - member authentication, short-lived token minting, and shortcode rendering that fails closed.

29.2 Retrieval and language intelligence

- **SQLite with FTS5** - the single-file knowledge store and keyword index.
- **NumPy** - cached embedding matrix and cosine scoring.
- **Reciprocal Rank Fusion** - combination of dense and keyword rankings.
- **Local sentence-transformer model** - 384-dimensional embeddings, the zero-cost production default.
- **Hosted embedding API** - 768-dimensional embeddings, the dependency-free alternative.
- **Gemini Flash** - grounded answer generation and, when enabled, grounded restricted web search.

29.3 Ingestion

- **Shared extraction module** - text, Markdown, HTML, PDF, and Word to clean text, with no network access.
- **Shared chunker and synchronizer** - word-boundary chunking, content hashing, and atomic snapshot synchronization.
- **Four connectors** - public transcript feed, Drive service account, membership API or export, course API or export.

29.4 Operations

- **systemd** - hardened service unit plus re-index and backup timers, all dedicated to this application.
- **nginx** - a new additive server block on its own hostname, proxying to a localhost port.
- **SQLite online backup** - consistent snapshots with configured retention pruning.

- **Allowlist release builder** - secret-audited client archive.
- **Docker** - verified optional packaging; not required by the runtime.

30. Current Status Summary

The JWALKER Knowledge Assistant is implemented and locally verified as a single-process, file-backed, hybrid-retrieval assistant with grounded answers, an honest refusal path, signed members-only access, four content connectors with atomic synchronization, a fail-closed restricted web search, isolated deployment templates, scheduled re-indexing and backups, and a secret-audited handover package.

The packaged knowledge base contains 228 chunks from 10 distinct video URLs. Local connector, retrieval, security, HTTP, migration, backup, static-analysis, container, and release-audit gates all pass on the 15 July 2026 verification run.

Production is not complete, and the project does not claim otherwise. Live deployment, live source pulls for three of the four connectors, and the production WordPress flow are blocked on client-controlled server access, content-platform access, and the outstanding restricted-search decisions. A development credential exposed during container validation requires rotation before any further provider use.

The strongest engineering qualities of the current system are the ordering guarantees around synchronization, the fail-closed posture of every gate (web search, production configuration, plugin rendering, release packaging), and the separation between an editable persona and grounding rules that cannot be edited away.

The strongest remaining needs are live production proof, credential rotation, an independent fresh-context review, a PHP validation gate, a golden retrieval set to protect evidence quality across future tuning, and shared-store rate limiting before any multi-worker deployment.

Appendix A - Representative Request Traces

A.1 Grounded member question

signed token verified -> rate limit ok -> question length ok

-> query embedded -> dense + keyword -> RRF fused

-> passages above relevance floor -> context assembled

-> persona + locked rules -> generated answer

-> sources deduplicated -> grounded = true

A.2 Off-topic question

signed token verified -> query embedded -> retrieval returns best candidates

-> every candidate below relevance floor -> passage set empty

-> configured refusal returned, no model call, no sources

-> grounded = false

A.3 Restricted web fallback

no passage clears the floor -> web search active?

-> enabled + keyword allowed + sites valid -> site-scoped query

-> citations re-validated against allowlist

-> answer prefixed as web-sourced -> grounded = false

A.4 Expired member token

Authorization header present -> signature valid -> expiry in the past

-> 401 returned -> no embedding, retrieval, or generation

A.5 Connector re-run with an edit and a deletion

discovery complete -> extraction complete -> hashes compared

-> unchanged documents skipped -> changed document embedded

-> single transaction: stale + replaced rows deleted, replacements inserted

-> commit -> counts reported for changed, unchanged, and deleted

A.6 Connector discovery failure

discovery raises -> no transaction opened -> no embedding billed

-> knowledge base unchanged -> failure reported to the operator

Appendix B - Safe Go-Live Checklist

1. Rotate the exposed development credential and configure a new production credential through the server environment only.
2. Take a read-only inventory of the target server and record the six existing agents' state, ports, resources, and proxy configuration.
3. Confirm the intended local port is unused and resources are sufficient; stop if either check fails.
4. Render deployment templates from configuration and install only the newly rendered units under a dedicated user and directory.
5. Validate the proxy configuration before reload; add a new server block on its own hostname and never edit an existing one.
6. Set the production environment marker and confirm the engine refuses to start on any unsafe setting.
7. Set the exact site origin for cross-origin access; the wildcard must not survive into production.
8. Install the plugin, define its three configuration constants, and confirm secret and lifetime parity with the engine.
9. Verify a logged-out visitor receives no widget, a logged-in member can chat, and an expired token is rejected.
10. Confirm the ingestion route requires its separate secret and that burst chat requests return 429.
11. Run each configured connector live, then run it a second time to prove update, delete, and idempotency behavior.
12. Confirm the scheduled re-index and backup jobs run, and perform one restore rehearsal.
13. Hash every deployed file against its local counterpart and confirm they match.
14. Re-check all six pre-existing agents against the inventory taken in step 2.
15. Configure provider budget caps and alerts on the account owner's side.

Appendix C - Glossary

- **RAG** - retrieval-augmented generation: retrieve supporting passages first, then generate an answer constrained to them.
- **Dense retrieval** - ranking by semantic vector similarity between the query and stored document embeddings.
- **FTS5** - SQLite's full-text search extension, used here as the keyword ranking arm.
- **RRF (Reciprocal Rank Fusion)** - a rank-combination method giving each candidate a score based on its position in each input list, used to merge dense and keyword rankings.
- **Relevance floor** - the minimum cosine score a passage must reach to be treated as relevant; below it, the refusal path runs.

- **Grounded** - a response flag meaning the answer was produced from passages that cleared the relevance floor.
- **Refusal guardrail** - the configured message returned, without a model call, when nothing relevant was retrieved.
- **Persona** - the editable voice and tone portion of the system prompt, separate from the locked grounding rules.
- **Source key** - a stable per-document identity within one source, which makes edit and delete synchronization possible.
- **Content hash** - a digest of a document's extracted text, used to skip unchanged documents without re-embedding them.
- **Atomic synchronization** - replacing and deleting rows for one source inside a single transaction, after all discovery, extraction, and embedding have already succeeded.
- **Additive ingestion** - adding new items by stable identity without treating the feed response as a complete snapshot; used where the source exposes only a recent window.
- **Fail closed** - defaulting to the safe outcome when configuration is missing or a check cannot be satisfied.
- **Stub mode** - retrieval-only operation with no generation provider configured, used to keep the pipeline testable without a credential.
- **Allowlist release build** - packaging from an explicit list of permitted files rather than excluding known-bad ones.
- **Parity check** - proving that a deployed file is byte-identical to the reviewed local artifact.

Appendix D - Review Checklist

For client and product review:

- Confirm the members-only boundary, the grounding promise, and the refusal behavior match expectations.
- Confirm which sources can refresh automatically and which require manual re-export, and accept that boundary explicitly.
- Confirm the restricted-search keyword list, site list, and mode decision.
- Confirm that generated answers remain subject to human review for claims and currency.
- Confirm that locally verified and production-proven are being reported as different states.

For engineering review:

- Trace one question from authentication through embedding, fusion, relevance gating, generation, and citation.
- Verify the relevance floor matches the configured embedding provider, and that the index was rebuilt after any provider change.

- Verify that synchronization completes all discovery, extraction, and embedding before opening the transaction.
- Verify that additive and snapshot sources are not confused with one another.
- Verify that the fused rank score is never used as a relevance threshold.
- Re-run the full local gate set after any change to retrieval, connectors, configuration, or dependencies.

For security and operations review:

- Confirm the signing secret never appears in HTML, JavaScript, logs, exports, or documentation.
- Confirm the ingestion secret is separate from the member secret and that both are set in production.
- Confirm production configuration validation rejects a wildcard origin and any missing secret.
- Confirm every restricted-search citation is re-validated against the allowlist rather than trusted from the provider.
- Confirm no command that expands an environment file is run with real values.
- Confirm the release archive passes its forbidden-path audit and independent secret scan.
- Confirm deployment was additive, parity was hashed, and every pre-existing agent matches its inventory.

Developed by Zahidul Islam

Backend and AI engineering - grounded retrieval, fail-closed access control, source synchronization integrity, cost-disciplined architecture, and client-safe delivery.